Frogger game setup steps

10/15/2026 21:52:39
https://claude.ai/chat/78020f58-bc85-4524-9577-3348a692e5b6

https://github.com/SETAPESU26/01_frogger tell me all the steps to do

Here is the full plan, combining the lab handout and the repo's README....

I finished all 4 tasks across 3 files: `collisions.py`, `game_engine.py` and...

PS C:\Users\clutc\01_frogger\frogger> git clone...

You have two small problems here, and both are easy to fix. ## Problem...

Next is copying in the fixed files, then recording the "after" video. ## 1....

Lab_4_VibeCoding_Student_handout.pdf

https://github.com/SETAPESU26/01_frogger tell me all the steps to do

10/15/2026, 2:46:41 PM

Planning a beginner-friendly lab setup guide.

- Retrieving the lab's repository for setup instructions.
- GitHub - SETAPESU26/01_frogger (github.com)
- Planning a beginner-friendly lab setup guide.
- Done

Here is the full plan, combining the lab handout and the repo's README. The game is a Pygame Frogger with 1 bug and 3 features to add.

Part A: Setup (about 5 min)

1. **Clone the repo.** Cloning is safer than forking, since you must not raise a PR to SETAPESU26.

```
git clone https://github.com/SETAPESU26/01_frogger.git
cd 01_frogger/frogger
```

2. **Install dependencies and run it.** Python 3.10+ is needed.

```
pip install -r requirements.txt
python main.py
```

3. **Play once.** Use the arrow keys to hop and R to restart. Notice the bug: the frog sometimes passes through a car that is clearly touching it.
4. **Record the "before" video (10 seconds).** Show the frog going through a vehicle without dying, ideally in a fast lane.

Part B: Prompt the AI, one task at a time

Open Claude (or ChatGPT) and keep it all in **one chat**, because you need to export the full history later. Paste the relevant files (`collisions.py`, `game_engine.py`, `frog.py`, `vehicle.py`, `renderer.py`) at the start so the AI sees your actual code. The goal is to finish each task in 3 to 4 prompts or fewer, so be specific.

Task 1: Fix collision detection (bug)
Prompt idea: *"Here is my Frogger code. Collision detection compares grid columns, so hits are missed. Change it to use real rectangle overlap between the frog and each vehicle, using their actual positions and sizes. Only fix collisions. Do not add lives yet."*
Test it by walking into cars in the fast lanes. Every overlap should be detected.

Task 2: Lives and respawn
Prompt idea: *"Add 3 lives. On a collision, subtract one life, reset the frog to the start, and show remaining lives on screen. When lives reach 0, show 'Game Over'. Also make the hit visible, for example a short red flash or a brief pause of about half a second before the respawn, so it is not an instant teleport."*
The README stresses that the hit moment must be visible, so check this carefully.

Task 3: Goal and score
Prompt idea: *"When the frog reaches the goal zone at the top, increase the score, end the game, and show 'You Won!'. Show the current score during play."*

Task 4: 30-second timer
Prompt idea: *"Add a 30-second countdown per attempt and display it. It starts when the attempt begins. If it hits zero before the goal, treat it as a failed attempt: lose a life and reset the frog and timer, and if that was the last life, game over. Reaching the goal in time is still a win. Pressing R must reset the timer too."*

Then check the "Expected Behavior" list yourself:

- The frog cannot move below its starting row.
- R restarts everything: lives, score, timer, game state.
- Game Over appears after 3 lives are lost.
- Timeout costs a life.

If something is off, tell the AI exactly what you saw and ask for the fix. The README wants you to critically review the code, not just paste it, so briefly read what the AI changed.

Part C: Record and export

1. **Record the "after" video (10 seconds).** Ideally show a collision with the life counter dropping, the timer counting down, and a win (or Game Over).
2. **Export the chat history** as a PDF or doc. The README also accepts a shareable chat link. Doing both is safest.

Part D: Push to your own repo

1. Create a new repo in your personal GitHub account (or use your existing Lab repo).
2. Make a **Lab-4** folder and put in it:
   - the before and after videos
   - the updated code (the `frogger` folder)
   - the chat history PDF or doc
3. Point your clone at your repo and push. Here `<your-username>` and `<your-lab-repo>` are placeholders for your own repo:

   ```
   git remote set-url origin https://github.com/<your-username>/<your-lab-repo>.git
   git add .
   git commit -m "Lab 4: Frogger vibecoding"
   git push -u origin main
   ```

   If you used an existing Lab repo instead, copy the files into its `Lab-4` folder and push from there.
4. **Do not open a PR** to SETAPESU26.

Quick checklist

- [ ] Before video (10 s)
- [ ] 4 tasks done in one chat
- [ ] After video (10 s)
- [ ] Chat exported as PDF or doc, plus link if possible
- [ ] Everything inside `Lab-4/` in your own repo

- [ ] No PR to the main repo

One tip: do the tasks in order, because Task 4 depends on the lives system from Task 2. Want me to draft the exact first prompt for Task 1 once you paste your `collisions.py`?

10/15/2026, 2:46:56 PM

Checking file availability and confirming the renderer tests pass.

- Coding the game loop, state transitions, and hit flash logic.
- Running command
- Running command
- Untangling a file-move error in the project structure.
- Running command
- Checking file availability and confirming the renderer tests pass.
- Running command
- Presenting file(s)...
- Done

I finished all 4 tasks across 3 files: `collisions.py`, `game_engine.py` and `renderer.py`. `frog.py`, `vehicle.py` and `main.py` stay unchanged. I ran a headless test of collision, timeout, game over, win and R-restart, and all of them behaved correctly. I haven't played it in a real window, so you should try that yourself.

What changed

- **Task 1 (collision bug):** `collisions.py` now checks whether the frog's rectangle and each vehicle's rectangle actually overlap. The old code compared grid columns, which is why fast cars slipped through.
- **Task 2 (lives and respawn):** You start with 3 lives. A hit makes the frog blink red/white for about 0.8 seconds and shows "Hit!". After that the frog respawns and the timer resets. At 0 lives you get "Game Over".
- **Task 3 (goal and score):** Reaching the top row gives 100 points plus 10 per second left, then shows "You Won!" with your score. The score is always shown on screen.
- **Task 4 (30-second timer):** The timer is shown top right and turns red under 10 seconds. If it hits 0, you lose a life, the same as a collision, and the message says "Time's up!". Reaching the goal in time is always a win. R resets everything, including the timer.

Movement is locked while the game is paused or over, so the frog can't wander during the hit flash. Only R works then.

Putting it on GitHub (PowerShell)

Replace `YOURUSERNAME` with your GitHub username. In PowerShell, use `py` if `python` doesn't work.

1. Clone your fork

```
cd ~\Desktop
git clone https://github.com/YOURUSERNAME/01_frogger.git
cd 01_frogger
```

2. Record your "before" video first. Do this now, before copying the fixed files, because the old code has the bug you need to show.

```
cd frogger
pip install -r requirements.txt
python main.py
```

Record 10 seconds of the frog passing through a car. On Windows, `Win + G` opens the Xbox Game Bar, which can record.

3. Copy in my 3 fixed files. Replace these in `game\` with the downloaded versions: `game\collisions.py`, `game\game_engine.py`, `game\renderer.py`

4. Run it and record the "after" video (10 seconds). Show a hit with the lives dropping, the timer counting down, and a win or Game Over.

5. Make the Lab-4 folder. Go back to the repo root and put in your videos and chat PDF:

```
cd ..
mkdir Lab-4
Copy-Item -Recurse frogger Lab-4\code
# then drag before.mp4, after.mp4 and your chat PDF into the Lab-4
folder
```

6. Push

```
git add .
git commit -m "Lab 4: Frogger fixes and features"
git push origin main
```

If Git asks you to sign in, use the browser pop-up. If it says `master` instead of `main`, push with `git push origin HEAD`.

Don't open a Pull Request. Your fork's page may show a "Contribute" button; ignore it.

Last bits

- **Chat history:** this whole chat counts as your history. Export it as a PDF (or use the share link) and add it to `Lab-4`.
- **Understand the code:** the lab says to critically review what the AI wrote, so spend a few minutes reading `game_engine.py`, especially the `update()` method.

collisions
Py

game_engine
Py

renderer
Py

10/15/2026, 9:16:48 PM

PS C:\Users\clutc\01_frogger\frogger> git clone https://github.com/Tharunyah/01_frogger.git
Cloning into '01_frogger'...
remote: Repository not found.
fatal: repository 'https://github.com/Tharunyah/01_frogger.git/' not found
PS C:\Users\clutc\01_frogger\frogger> git clone https://github.com/Tharunyah/01_frogger.git
Cloning into '01_frogger'...
remote: Enumerating objects: 20, done.
remote: Counting objects: 100% (20/20), done.
remote: Compressing objects: 100% (16/16), done.
remote: Total 20 (delta 1), reused 20 (delta 1), pack-reused 0 (from 0)
Receiving objects: 100% (20/20), 7.65 KiB | 1.09 MiB/s, done.
Resolving deltas: 100% (1/1), done.
PS C:\Users\clutc\01_frogger\frogger> pip install -r requirements.txt
pip : The term 'pip' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the name, or if a path
was included, verify that the path is correct and try again.
At line:1 char:1

- pip install -r requirements.txt
- ```
  + CategoryInfo          : ObjectNotFound: (pip:String) [], CommandNotFoundException
  + FullyQualifiedErrorId : CommandNotFoundException
  ```

PS C:\Users\clutc\01_frogger\frogger>

10/15/2026, 9:23:25 PM
  ```

Untangling a repository cloned into the wrong nested folder.

Untangling a repository cloned into the wrong nested folder.

Done

You have two small problems here, and both are easy to fix.

Problem 1: the clone landed inside another folder

Your first clone failed because the fork didn't exist yet. The second one worked, but you ran it from inside `C:\Users\clutc\01_frogger\frogger`, so the repo ended up in a nested folder. To avoid confusion, clone it again into a clean place:

```
cd ~
git clone https://github.com/Tharunyah/01_frogger.git frogger-lab
cd frogger-lab\frogger
dir
```

`dir` should show `main.py`, `requirements.txt` and a `game` folder. From now on, use only this `frogger-lab` folder and ignore the older ones.

Problem 2: `pip` isn't found

This usually means Python isn't installed, or it isn't on your PATH. Check which command works:

```
py --version
python --version
```

- **If `py --version` prints a version** (3.10 or higher), use `py` for everything:
  ```
  py -m pip install -r requirements.txt
  py main.py
  ```
- **If both say "not recognized"**, install Python:
  1. Download it from https://www.python.org/downloads/
  2. In the installer, **tick "Add python.exe to PATH"** at the bottom of the first screen, then click Install.
  3. Close PowerShell completely and open a new one. Then run:
     ```
     cd ~\frogger-lab\frogger
     python -m pip install -r requirements.txt
     ```

```
python main.py
```

If `requirements.txt` isn't found, run `py -m pip install pygame` (or `python -m pip install pygame`) instead. Pygame is the only thing the game needs.

Next

Once the game window opens, that's your **"before" video**. Record 10 seconds of the frog passing through a car, then come back and tell me when you're at the copy-the-files step. If you hit any other error, paste it here and I'll sort it out.

10/15/2026, 9:23:35 PM

Next is copying in the fixed files, then recording the "after" video.

1. Copy the 3 fixed files

Download `collisions.py`, `game_engine.py` and `renderer.py` from my last reply. They probably went to your Downloads folder. Then run:

```
cd ~\frogger-lab\frogger
Copy-Item ~\Downloads\collisions.py game\collisions.py -Force
Copy-Item ~\Downloads\game_engine.py game\game_engine.py -Force
Copy-Item ~\Downloads\renderer.py game\renderer.py -Force
```

If a file has a name like `collisions (1).py`, change the name in the command to match.

2. Run it

```
py main.py
```

Use `python main.py` if that's what worked for you before. Quickly check that:

- "Lives", "Score" and "Time" show at the top.
- Walking into a car makes the frog blink red and costs a life.
- Reaching the top row shows "You Won!".

3. Record the "after" video (10 seconds)

Try to show these in one take:

1. Hit a car, and the lives drop from 3 to 2.
2. The timer counting down.
3. A win (or Game Over if you lose all lives).

4. Make the Lab-4 folder

```
cd ~\frogger-lab
mkdir Lab-4
Copy-Item -Recurse frogger Lab-4\code
```

Then drag your two videos into `Lab-4`. Rename them `before.mp4` and `after.mp4` if you like. Also export this chat as a PDF and put it in the same folder.

5. Push

```
git add .
git commit -m "Lab 4: Frogger fixes and features"
git push origin main
```

If it asks you to sign in, use the browser pop-up. Afterwards, open `github.com/Tharunyah/01_frogger` and check that `Lab-4` is there with the videos, code and chat PDF. Don't click "Contribute" or open a Pull Request.

If any step errors, paste the output here.

10/15/2026, 9:25:31 PM